

LAPORAN ALGORITMA DAN STRUKTUR DATA IOT NETWORK MONITORING DAN ALERT MANAGEMENT SYSTEM

Dosen Pengampu:

Dr. Eng. Ir. Aji Ery Burhandenny, ST., M.AIT.



Disusun oleh:

Muhamad Padang Tazaka	(25051030121)
Nauval Fadhlurohman As-Sidiq	(25051030120)
Taamir Ghalib Athallah	(25051030092)
Muhamad Fahdillah Setyo Wibowo	(25051030081)

**PROGRAM STUDI TEKNIK ELEKTRO
DEPARTEMEN PENDIDIKAN TEKNIK ELEKTRO
FAKULTAS TEKNIK
UNIVERSITAS NEGERI YOGYAKARTA**

2026

DAFTAR ISI

DAFTAR ISI	2
BAB 1 PENDAHULUAN	3
A. Latar Belakang Masalah	3
B. Tujuan Sistem	3
C. Batasan Implementasi	3
D. Diagram Arsitektur Sistem	4
BAB II PERANCANGAN SISTEM	5
A. Pemilihan Struktur Data dan Justifikasi	5
B. Diagram Interaksi Antar Modul	6
C. Pseudocode Algoritma Utama	7
BAB III IMPLEMENTASI	8
A. Penjelasan Kode Per Modul	8
B. Screenshot CLI Berjalan	11
C. Contoh Input/Output Nyata	11
BAB IV ANALISIS BIG-O	14
A. Analisis Kompleksitas Waktu dan Ruang Setiap Fungsi utama	14
B. Tabel Ringkasan	15
C. Perbandingan Teoritis vs Eksperimen	15
BAB V HASIL EKSPERIMEN	16
A. Tabel Runtime Untuk 3+ Ukuran Data	16
B. Grafik Tren	17
C. Diskusi	17
BAB VI JAWABAN PERTANYAAN ANALISIS	18
BAB VII PENUTUP	20
A. KESIMPULAN	20
LAMPIRAN	21

BAB 1

PENDAHULUAN

A. Latar Belakang Masalah

Perkembangan pesat teknologi Internet of Things (IoT) saat ini telah menyentuh berbagai aspek kehidupan dan industri. Hal ini berbanding lurus dengan peningkatan drastis dalam jumlah perangkat yang saling terhubung di dalam sebuah jaringan. Ratusan hingga ribuan sensor dan perangkat cerdas secara terus-menerus mengirimkan data secara real-time, sehingga sangat menuntut adanya infrastruktur jaringan yang tangguh dan responsif untuk mengelola arus informasi tersebut.

Namun, jaringan berskala besar ini sering kali rentan terhadap berbagai masalah operasional dan teknis. Tingginya volume data yang masuk kerap menyebabkan latensi (keterlambatan) jaringan, lambatnya pemrosesan pesan peringatan (alert), serta kesulitan bagi sistem dalam melacak status dan riwayat kerusakan pada masing-masing perangkat. Tanpa sistem manajemen yang terstruktur dan efisien, penumpukan data peringatan ini akan memperlambat proses pemantauan dan pada akhirnya dapat mengakibatkan kegagalan sistem yang fatal jika ada kondisi kritis yang terlewatkan.

Oleh karena itu, sebuah sistem manajemen alert dan pemantauan jaringan yang dioptimalkan menggunakan struktur data yang tepat sangat dibutuhkan untuk menyelesaikan masalah tersebut. Dengan mengimplementasikan algoritma pencarian rute tercepat, sistem antrian prioritas untuk peringatan, serta rekam jejak perangkat yang efisien, sistem ini dirancang untuk memastikan setiap masalah pada jaringan IoT dapat dideteksi, diprioritaskan, dan ditangani dengan secepat mungkin.

B. Tujuan Sistem

Sistem ini dikembangkan dengan tujuan-tujuan spesifik sebagai berikut:

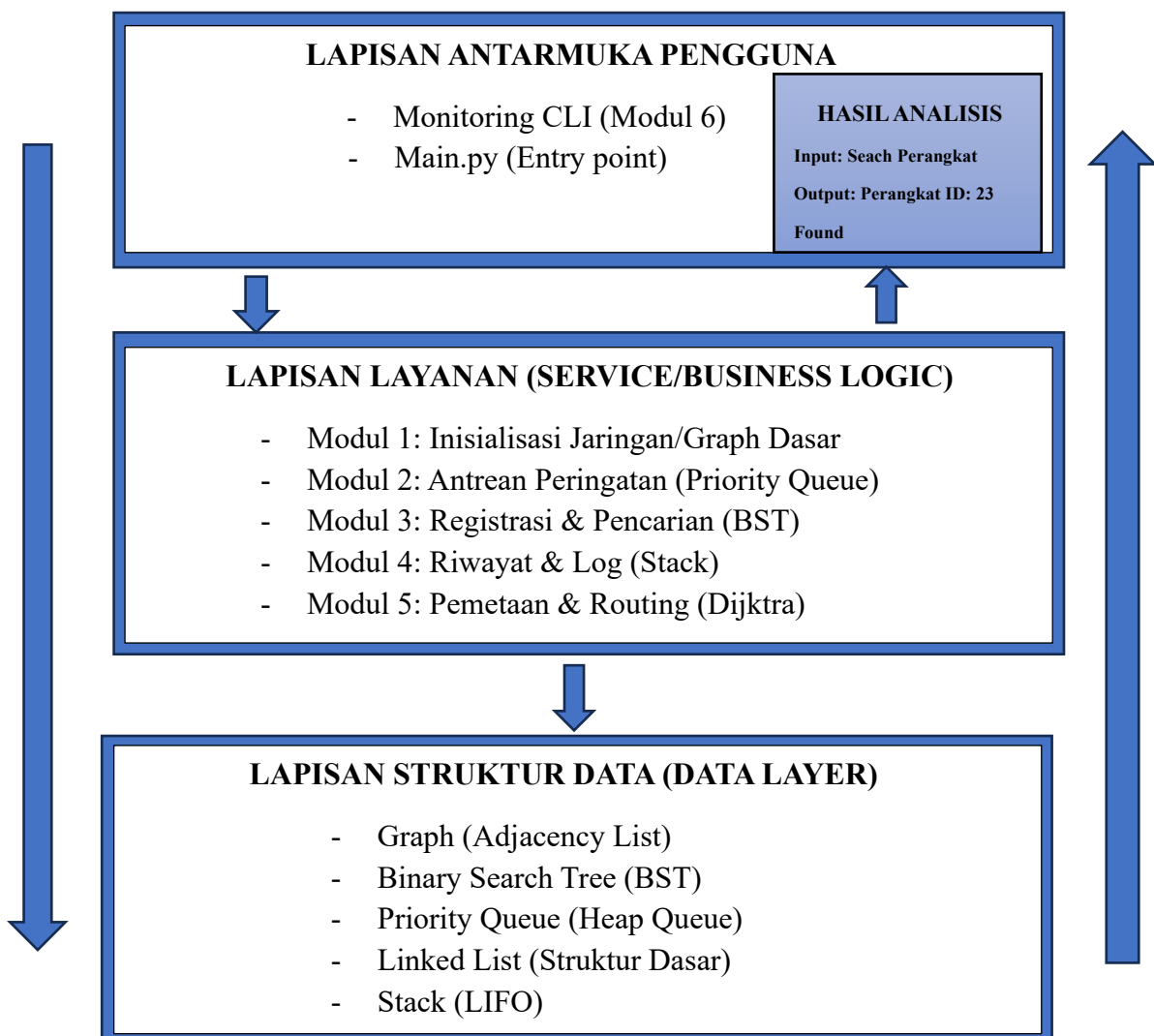
1. Mengelola dan memprioritaskan antrian pesan peringatan (alert) yang masuk berdasarkan tingkat kekritisannya secara efisien menggunakan struktur data Priority Queue.
2. Memfasilitasi proses registrasi dan pencarian data perangkat IoT dalam jaringan secara cepat dan terstruktur menggunakan Binary Search Tree (BST).
3. Menyimpan riwayat peringatan pada masing-masing perangkat menggunakan Stack (LIFO) guna mempermudah proses audit dan pelacakan status terakhir perangkat.
4. Mensimulasikan topologi jaringan antar-perangkat dan menentukan rute komunikasi dengan latensi terendah (Shortest Path) menggunakan struktur data Graph dan algoritma Dijkstra.

C. Batasan Implementasi

Batasan ruang lingkup (scope) dalam pengembangan sistem ini ditetapkan agar fokus evaluasi proyek tetap berada pada analisis efisiensi struktur data dan algoritma. Batasan implementasi tersebut meliputi:

1. Bahasa Pemrograman: Sistem ini dikembangkan sepenuhnya menggunakan bahasa pemrograman Python standar.
2. Implementasi Struktur Data: Sistem menggunakan implementasi struktur data murni yang dibangun dari awal (from scratch), meliputi Linked List, Stack, Queue, Binary Search Tree, dan Graph. Sistem tidak bergantung pada built-in library Python untuk komponen intinya (pengecualian pada array dasar dan modul heapq murni untuk optimasi algoritma Dijkstra).
3. Penyimpanan Data: Sistem beroperasi sepenuhnya di dalam memori utama (in-memory) dan tidak terintegrasi dengan database eksternal. Data perangkat dan peringatan (mock data) dibangkitkan secara acak menggunakan fungsi seed (seed=23) untuk memastikan konsistensi hasil saat tahap pengujian/eksperimen.
4. Antarmuka Pengguna: Interaksi antara pengguna dan sistem dilakukan secara eksklusif melalui Antarmuka Baris Perintah (Command Line Interface / CLI), tanpa menggunakan Graphical User Interface (GUI).
5. Simulasi Jaringan: Sistem ini mensimulasikan topologi jaringan dan status perangkat keras (IoT) secara logis di tingkat perangkat lunak, dan tidak terhubung secara langsung dengan perangkat keras fisik maupun sensor sebenarnya.

D. Diagram Arsitektur Sistem



BAB II

PERANCANGAN SISTEM

A. Pemilihan Struktur Data dan Justifikasi

Dalam pengembangan Sistem Manajemen Peringatan dan Pemantauan Jaringan IoT ini, pemilihan struktur data yang tepat sangat krusial untuk menjamin kinerja yang cepat (real-time) dan hemat memori. Berikut adalah rincian struktur data yang digunakan beserta justifikasi pemilihannya:

1. Graph (Adjacency List)

- a. Tujuan Penggunaan: Digunakan untuk merepresentasikan topologi jaringan secara keseluruhan, di mana perangkat IoT bertindak sebagai simpul (node/vertex) dan jalur komunikasi antar-perangkat bertindak sebagai sisi (edge) yang memiliki bobot berupa latensi jaringan.
- b. Justifikasi: Implementasi Graph menggunakan Adjacency List (Daftar Ketetanggaan) dipilih karena jaringan IoT di dunia nyata umumnya bersifat sparse graph (graf renggang), di mana satu perangkat tidak terhubung langsung ke semua perangkat lain. Adjacency List jauh lebih hemat memori dengan kompleksitas ruang $O(V + E)$ dibandingkan Adjacency Matrix yang membutuhkan $O(V^2)$. Struktur ini juga sangat optimal saat dipadukan dengan algoritma Dijkstra untuk operasi pencarian rute terpendek (Shortest Path).

2. Priority Queue (berbasis Heap)

- a. Tujuan Penggunaan: Digunakan pada Modul 2 untuk mengelola antrean pesan peringatan (alert) yang masuk ke dalam sistem terpusat.
- b. Justifikasi: Dalam sistem IoT, alert tidak bisa diproses hanya berdasarkan urutan waktu masuk (FIFO/First-In-First-Out). Peringatan yang kritis (seperti indikasi kebakaran atau offline) harus diproses lebih dahulu dibandingkan peringatan biasa (seperti update firmware). Priority Queue memungkinkan penyisipan (push) dan pengambilan data prioritas tertinggi (pop) dengan kompleksitas waktu yang sangat cepat yaitu $O(\log N)$, jauh lebih efisien dibandingkan menyortir array biasa yang memakan waktu $O(N \log N)$.

3. Binary Search Tree (BST)

- a. Tujuan Penggunaan: Digunakan pada Modul 3 sebagai buku register utama untuk mendaftarkan dan melakukan pencarian perangkat (Device ID) secara spesifik di dalam jaringan.
- b. Justifikasi: Jaringan IoT memiliki karakteristik dinamis di mana perangkat sering ditambah atau dicari statusnya. Struktur Binary Search Tree (BST) dipilih karena menawarkan efisiensi pencarian, penyisipan, dan penghapusan data dengan kompleksitas rata-rata $O(\log N)$. Hal ini memberikan keunggulan performa yang jauh

lebih cepat dibandingkan pencarian sekuensial pada Array atau Linked List tunggal yang membutuhkan waktu $O(N)$.

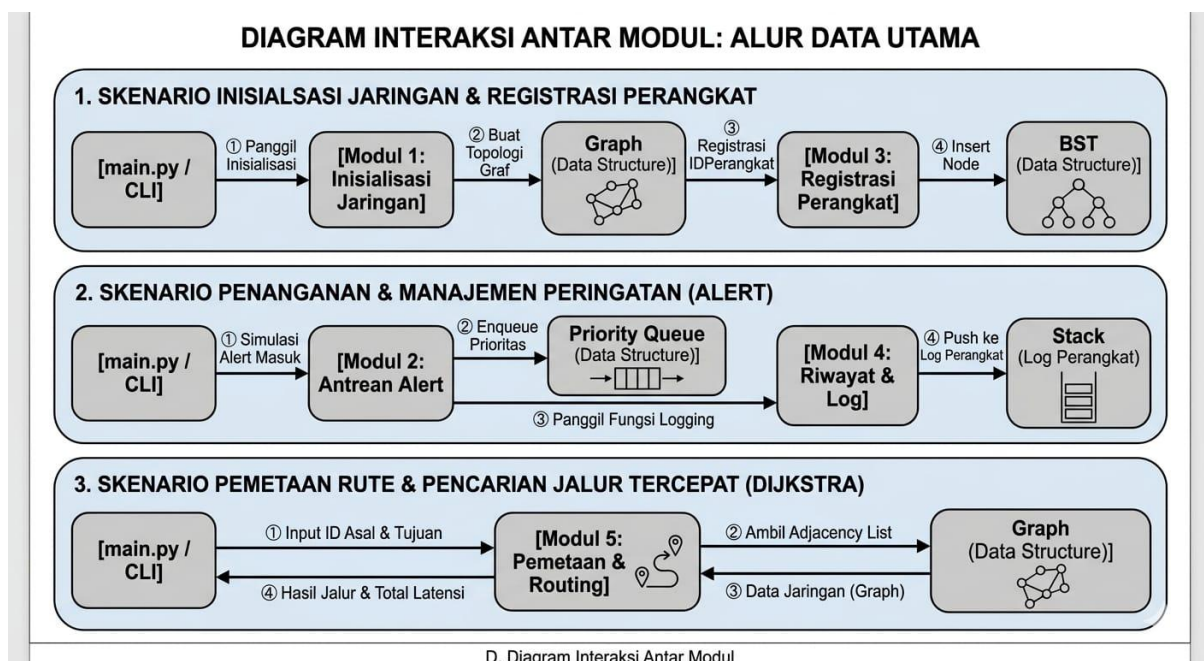
4. Stack (LIFO)

- Tujuan Penggunaan: Digunakan pada Modul 4 untuk menyimpan riwayat (log/history) kerusakan atau perubahan status pada masing-masing perangkat IoT.
- Justifikasi: Prinsip Last-In-First-Out (LIFO) pada Stack sangat ideal untuk sistem logging dan audit. Saat sistem atau admin jaringan perlu melakukan pengecekan (backtracking) atau mencabut status peringatan terakhir (rollback), alert yang paling terakhir dimasukkan adalah yang paling pertama ditarik keluar. Eksekusi penambahan (push) dan pengambilan (pop) pada Stack berjalan secara instan dengan kompleksitas waktu konstan yaitu $O(1)$.

5. Linked List (Sebagai Fondasi Dinamis)

- Tujuan Penggunaan: Digunakan sebagai struktur data dasar (building block) untuk membangun struktur antrian dan tumpukan (Stack/Queue) murni.
- Justifikasi: Menggunakan Linked List sebagai fondasi memungkinkan sistem untuk mengalokasikan memori secara dinamis (dynamic memory allocation). Sistem IoT sering kali menerima lonjakan data yang tidak terprediksi. Berbeda dengan Array statis yang ukuran memorinya harus dipesan di awal dan rentan penuh, Linked List dapat membesar atau mengecil secara fleksibel seiring dengan jumlah data alert atau log yang diproses, sehingga mencegah terjadinya kebocoran atau pemborosan memori.

B. Diagram Interaksi Antar Modul



C. Pseudocode Algoritma Utama

Untuk memberikan gambaran logika tingkat tinggi (high-level logic) yang independen dari bahasa pemrograman Python, berikut adalah pseudocode untuk dua algoritma utama yang menjadi inti pemrosesan dalam sistem ini:

1. Algoritma Dijkstra (Modul 5: Pemetaan & Routing)

Algoritma ini bertugas mencari jalur komunikasi dengan latensi terendah antar-perangkat menggunakan struktur Graph dan Priority Queue untuk optimasi ekstraksi node dengan jarak minimum.

```
Fungsi CariRuteTercepat(Graph, StartNode, TargetNode):  
    // 1. Inisialisasi  
    Inisialisasi array Distances dengan nilai Tak Hingga ( $\infty$ ) untuk semua node  
    Distances[StartNode] = 0  
    Inisialisasi PriorityQueue PQ kosong  
    Masukkan (0, StartNode) ke dalam PQ  
    Inisialisasi array PreviousNodes untuk melacak jejak rute  
  
    // 2. Proses Pencarian  
    Selama PQ TIDAK kosong lakukan:  
        (CurrentDistance, CurrentNode) = Ekstrak nilai terkecil dari PQ  
  
        // Optimasi: Jika sudah mencapai target, proses dihentikan awal  
        Jika CurrentNode sama dengan TargetNode:  
            Berhenti dari perulangan (Break)  
  
        // Abaikan jika menemukan rute yang lebih lambat dari catatan  
        Jika CurrentDistance > Distances[CurrentNode]:  
            Lanjutkan ke iterasi berikutnya (Continue)  
  
        // Evaluasi setiap node tetangga  
        Untuk setiap (Tetangga, BobotLatensi) dari  
        Graph.GetNeighbors(CurrentNode) lakukan:  
            JarakBaru = CurrentDistance + BobotLatensi  
  
            // Relaksasi Edge: Jika ditemukan rute yang lebih cepat  
            Jika JarakBaru < Distances[Tetangga]:  
                Distances[Tetangga] = JarakBaru  
                PreviousNodes[Tetangga] = CurrentNode  
                Masukkan (JarakBaru, Tetangga) ke dalam PQ  
  
    // 3. Bangun ulang rute dari target ke start  
    Jalur = BangunJalur(PreviousNodes, TargetNode)  
  
    Kembalikan (Jalur, Distances[TargetNode])  
Akhir Fungsi
```

2. Algoritma Binary Search Tree - Insert (Modul 3: Registrasi)

Algoritma rekursif ini digunakan untuk menyisipkan data perangkat baru ke dalam pohon pencarian. Logika ini memastikan data selalu terurut sehingga proses pencarian (search) nantinya menjadi jauh lebih cepat.

```
Fungsi RegistrasiPerangkat(Node, DeviceID, DataPerangkat):
    // Jika pohon/cabang kosong, buat node baru di sini
    Jika Node adalah KOSONG:
        Kembalikan NodeBaru(DeviceID, DataPerangkat)

    // Jika ID lebih kecil, telusuri cabang sebelah kiri
    Jika DeviceID < Node.DeviceID:
        Node.Kiri = RegistrasiPerangkat(Node.Kiri, DeviceID, DataPerangkat)

    // Jika ID lebih besar, telusuri cabang sebelah kanan
    Atau Jika DeviceID > Node.DeviceID:
        Node.Kanan = RegistrasiPerangkat(Node.Kanan, DeviceID, DataPerangkat)

    // Jika ID sama, tolak duplikasi
    Lainnya:
        Cetak "Error: Perangkat dengan ID tersebut sudah terdaftar!"

    // Kembalikan node saat ini agar pohon tetap terhubung
    Kembalikan Node
Akhir Fungsi
```

BAB III

IMPLEMENTASI

A. Penjelasan Kode Per Modul

Pada tahap implementasi, sistem dibangun menggunakan bahasa pemrograman Python dengan paradigma modular. Pemisahan fungsi ke dalam beberapa modul ini dilakukan untuk mempermudah proses pemeliharaan kode (maintainability) dan debugging. Berikut adalah penjelasan teknis untuk masing-masing modul:

1. Modul 1: Inisialisasi Jaringan (Struktur Data: Graph)

Modul ini bertindak sebagai service layer yang menjembatani antarmuka pengguna (CLI) dengan struktur data utama IoTGraph. Modul ini mengelola operasi dasar jaringan seperti penambahan perangkat (simpul) dan pembentukan jalur komunikasi (link) antar-perangkat beserta bobot latensinya dengan kompleksitas waktu instan $O(1)$. Keunggulan analitik dari modul ini terletak pada fitur `run_deteksi_isolasi`, yang secara aktif mengimplementasikan algoritma penelusuran Depth-First Search (DFS) berbasis Stack dengan kompleksitas $O(V+E)$. Fitur ini berfungsi untuk mengaudit topologi dan mendeteksi secara real-time apabila ada perangkat yang terputus (terisolasi) dari gateway pusat. Selain itu, modul ini juga dilengkapi fungsi untuk mencetak ringkasan topologi yang menampilkan metrik degree (jumlah tetangga) dari setiap perangkat di dalam jaringan.

2. Modul 2: Antrean Peringatan (Struktur Data: Priority Queue)

Modul ini berfungsi sebagai lapisan layanan untuk menangani manajemen pesan peringatan (alert) jaringan secara prioritas. Implementasinya memanfaatkan kelas `AlertPriorityQueue` dari paket struktur data lokal. Berbasis pada karakteristik sistem, setiap objek alert memiliki atribut tingkat kekritisan yang dipetakan menjadi tiga tingkatan: CRITICAL (1), WARNING (2), dan INFO (3). Berdasarkan dokumentasi kode, operasi memasukkan data (`run_tambah_alert`) berjalan dengan kompleksitas waktu worst-case $O(n)$ karena sistem melakukan pengurutan posisi secara langsung di dalam antrean. Keunggulannya, proses pengambilan dan eksekusi peringatan tertinggi (`run_proses_alert`) dapat berjalan secara instan dengan kompleksitas konstan $O(1)$. Modul ini juga menyediakan fungsi pencetakan seluruh daftar antrean tertunda (pending alerts) dengan kompleksitas waktu $O(n)$.

3. Modul 3: Registrasi dan Pencarian (Struktur Data: Binary Search Tree)

Modul ini berfungsi sebagai lapisan layanan untuk manajemen pencatatan dan lookup data perangkat IoT secara terpusat melalui kelas `BSTRegistry`. Pada fungsi `run_register_device`, proses penambahan perangkat baru ke dalam pohon biner terurut dilengkapi dengan blok proteksi try-except untuk menangkap eksepsi `ValueError` guna mencegah terjadinya duplikasi ID perangkat di dalam sistem. Untuk mendukung pemantauan jaringan yang dinamis, modul ini menyediakan fungsi `run_search_device` dan `run_update_status` yang bertugas mencari perangkat serta memperbarui parameter operasionalnya (seperti status ONLINE/OFFLINE dan nilai bacaan sensor terakhir) dengan efisiensi kompleksitas waktu rata-rata $O(\log n)$. Terakhir, modul ini mengimplementasikan fungsi `run_list_devices` yang memanfaatkan metode penelusuran pohon secara inorder traversal (`inorder()`) untuk menarik data dari seluruh simpul pohon, sehingga sistem dapat menyajikan daftar perangkat secara terurut alfabetis berdasarkan ID dengan kompleksitas waktu $O(n)$.

4. Modul 4: Riwayat dan Log (Struktur Data: Stack)

Modul ini bertindak sebagai service layer untuk mengelola riwayat peringatan (alert history) secara terisolasi pada masing-masing perangkat menggunakan kelas `AlertStack` dengan prinsip Last-In-First-Out (LIFO). Pada fungsi `run_tambah_riwayat`, operasi penyisipan data (`push()`) didesain memiliki batasan kapasitas memori maksimal 20 peringatan per perangkat. Jika kapasitas terlampaui, sistem secara otomatis menghapus data paling usang di dasar tumpukan (bottom), sehingga operasi penyisipan yang normalnya $O(1)$ akan bergeser menjadi $O(n)$ pada kondisi worst-case. Keunggulan utama modul ini terletak pada fitur `run_rollback_status`. Fungsi ini tidak hanya mengeksekusi operasi `pop()` untuk membatalkan peringatan terakhir secara instan ($O(1)$), tetapi juga berinteraksi lintas-modul dengan objek Registry (BST) untuk memulihkan status perangkat kembali menjadi ONLINE. Untuk keperluan audit, modul ini menyediakan fungsi `run_lihat_riwayat` yang menelusuri data menggunakan metode `to_list()` guna menyajikan riwayat dari urutan terbaru hingga terlama secara sekuensial dengan kompleksitas $O(n)$.

5. Modul 5: Pemetaan dan Routing (Algoritma Dijkstra)

Modul ini merupakan inti analitik sistem untuk optimasi dan evaluasi performa jaringan. Pada fungsi `run_routing_optimal`, sistem mengimplementasikan Algoritma Dijkstra menggunakan bantuan modul `heapq` (Priority Queue) untuk mencari rute komunikasi dengan latensi terendah dari gateway ke seluruh perangkat secara dinamis dengan kompleksitas waktu $O((V+E) \log V)$. Selama pencarian rute, modul ini juga membangun Shortest Path Tree (SPT) untuk mengidentifikasi bottleneck link, yaitu jalur aktif dengan beban latensi tertinggi yang membutuhkan evaluasi perangkat keras. Selain itu, modul ini dilengkapi fitur `run_audit_latensi` yang mentransfer data jarak ke dalam struktur data dinamis Linked List kustom. Sistem kemudian menjalankan algoritma Selection Sort murni di atas Linked List tersebut untuk mengurutkan perangkat berdasarkan latensi kumulatifnya dengan kompleksitas $O(V^2)$. Pendekatan ini mendemonstrasikan penguasaan manipulasi pointer tanpa bergantung pada fungsi pengurutan bawaan bahasa pemrograman.

6. Antarmuka Pengguna (main.py / Monitoring CLI)

Modul ini bertindak sebagai Entry Point (titik awal) sekaligus orkestrator utama yang mengintegrasikan seluruh layanan struktur data ke dalam satu kesatuan sistem. Diimplementasikan menggunakan perulangan dinamis `while True`;, antarmuka berbasis Command Line Interface (CLI) ini secara aktif mendengarkan masukan dari pengguna dan menggunakan fungsi bawaan `split()` untuk mengurai (parsing) teks string menjadi instruksi dan argumen parameter. Keunggulan arsitektural dari modul ini adalah kemampuannya melakukan sinkronisasi lintas-modul secara real-time. Sebagai contoh, saat instruksi `ALERT_IN` dieksekusi, sistem secara otomatis melakukan orkestrasi pemanggilan Modul 2 (enqueue antrean), Modul 4 (push ke riwayat log), dan Modul 3 (pembaruan status node BST) dalam satu alur waktu yang sama. Guna menjamin keandalan sistem (reliability), seluruh proses di dalam antarmuka ini dibungkus menggunakan lapisan proteksi `try-except` level atas untuk mencegah aplikasi berhenti tiba-tiba (crash) ketika terjadi kesalahan masukan atau anomali data struktural.

B. Screenshot CLI Berjalan

```
=====
AUDIT LATENSI PERANGKAT (SELECTION SORT ON LINKED LIST)
=====
Daftar Perangkat Terurut Berdasarkan Latensi Kumulatif ke GATEWAY:
1. SENSOR_25 -> Total Latensi: 30 ms
2. SENSOR_39 -> Total Latensi: 33 ms
3. SENSOR_22 -> Total Latensi: 96 ms
4. SENSOR_26 -> Total Latensi: 99 ms
5. SERVER_3 -> Total Latensi: 110 ms
6. SENSOR_10 -> Total Latensi: 111 ms
7. SENSOR_17 -> Total Latensi: 112 ms
8. SENSOR_36 -> Total Latensi: 125 ms
9. SENSOR_30 -> Total Latensi: 134 ms
10. SENSOR_11 -> Total Latensi: 148 ms
11. SENSOR_31 -> Total Latensi: 165 ms
12. SENSOR_34 -> Total Latensi: 165 ms
13. SERVER_2 -> Total Latensi: 166 ms
14. SENSOR_19 -> Total Latensi: 169 ms
15. SENSOR_37 -> Total Latensi: 174 ms
16. SENSOR_16 -> Total Latensi: 178 ms
17. SERVER_4 -> Total Latensi: 190 ms
18. SENSOR_21 -> Total Latensi: 209 ms
19. SENSOR_13 -> Total Latensi: 212 ms
20. SENSOR_8 -> Total Latensi: 214 ms
21. SENSOR_27 -> Total Latensi: 220 ms
22. SENSOR_28 -> Total Latensi: 223 ms
23. SENSOR_23 -> Total Latensi: 226 ms
24. SENSOR_18 -> Total Latensi: 241 ms
25. SENSOR_33 -> Total Latensi: 256 ms
26. SENSOR_7 -> Total Latensi: 265 ms
27. SENSOR_12 -> Total Latensi: 269 ms
28. SENSOR_5 -> Total Latensi: 271 ms
29. SENSOR_6 -> Total Latensi: 297 ms
30. SENSOR_29 -> Total Latensi: 317 ms
31. SENSOR_24 -> Total Latensi: 321 ms
32. SERVER_1 -> Total Latensi: 328 ms
33. SENSOR_9 -> Total Latensi: 334 ms
34. SENSOR_20 -> Total Latensi: 340 ms
35. SENSOR_14 -> Total Latensi: 374 ms
36. SENSOR_15 -> Total Latensi: 392 ms
37. SENSOR_32 -> Total Latensi: 399 ms
38. SENSOR_38 -> Total Latensi: 414 ms
39. SENSOR_35 -> Total Latensi: 503 ms
=====
-> Kompleksitas Waktu: Big-O:  $O(V^2)$  - Selection Sort pada Linked List
```

```
-----
IoT-CLI> ALERT_IN SENSOR_25 1 Kebakaran_Ruang_Server
[ENQUEUE] Alert #2 dari 'SENSOR_25' (CRITICAL) masuk ke antrean.
-> Kompleksitas Waktu: Big-O:  $O(n)$  worst-case

[HISTORY: PUSH] Alert #2 berhasil disimpan ke riwayat 'SENSOR_25'.
-> Kompleksitas Waktu: Big-O:  $O(1)$  normal /  $O(n)$  worst-case

[UPDATE] Status 'SENSOR_25' diperbarui menjadi OFFLINE.
-> Kompleksitas Waktu: Big-O:  $O(\log n)$  rata-rata
-----
```

C. Contoh Input/Output Nyata

Untuk memberikan gambaran interaksi sistem secara tekstual, berikut adalah transkripsi simulasi masukan perintah dari pengguna beserta keluaran nyata yang dihasilkan oleh sistem monitoring jaringan IoT melalui Command Line Interface (CLI):

1. Skenario Audit Latensi dan Pengurutan Jaringan (Modul 5)

Skenario ini menunjukkan bagaimana sistem mengeksekusi perintah untuk menghitung jalur terpendek dari pusat kontrol (GATEWAY_0) menggunakan algoritma Dijkstra, kemudian mentransfer data tersebut ke dalam struktur data Linked List untuk diurutkan dari latensi terkecil ke terbesar menggunakan Selection Sort.

- Input dari pengguna:

AUDIT_LATENSI

- Output system:

[INFO] Menghitung jalur dari GATEWAY_0 untuk diaudit...

ROUTING OPTIMAL (DIJKSTRA) DARI: GATEWAY_0

-> SENSOR_1: 52 ms | Rute: GATEWAY_0 -> SENSOR_1
-> SENSOR_2: 124 ms | Rute: GATEWAY_0 -> SENSOR_1 -> SENSOR_2
-> SENSOR_22: 96 ms | Rute: GATEWAY_0 -> SENSOR_22
-> SENSOR_25: 30 ms | Rute: GATEWAY_0 -> SENSOR_25
-> SENSOR_39: 33 ms | Rute: GATEWAY_0 -> SENSOR_39
... (data rute Dijkstra dipotong untuk efisiensi halaman) ...

-> Kompleksitas Waktu: Big-O: $O((V+E) \log V)$

AUDIT LATENSI PERANGKAT (SELECTION SORT ON LINKED LIST)

Daftar Perangkat Terurut Berdasarkan Latensi Kumulatif ke GATEWAY:

1. SENSOR_25 -> Total Latensi: 30 ms
2. SENSOR_39 -> Total Latensi: 33 ms
3. SENSOR_1 -> Total Latensi: 52 ms
4. SENSOR_22 -> Total Latensi: 96 ms
5. SENSOR_2 -> Total Latensi: 124 ms
- ... (data urutan hasil Selection Sort dipotong) ...
39. SENSOR_35 -> Total Latensi: 503 ms

-> Kompleksitas Waktu: Big-O: $O(V^2)$ - Selection Sort pada Linked List

2. Skenario Manajemen Masukan Gangguan Jaringan (Modul 2, Modul 3, dan Modul 4)

Skenario ini mensimulasikan terjadinya anomali atau bahaya pada salah satu sensor yang terdaftar di dalam jaringan. Perintah ALERT_IN secara simultan memicu tiga operasi struktur data: penambahan antrean prioritas (Modul 2), pencatatan log riwayat Stack (Modul 4), dan pembaruan status perangkat di pohon BST menjadi OFFLINE (Modul 3).

- Input dari pengguna:

```
ALERT_IN SENSOR_25 1 Kebakaran_Ruang_Server
```

- Output system:

```
[ENQUEUE] Alert #1 dari 'SENSOR_25' (CRITICAL) masuk ke antrean.  
-> Kompleksitas Waktu: Big-O:  $O(n)$  worst-case
```

```
[HISTORY: PUSH] Alert #1 berhasil disimpan ke riwayat 'SENSOR_25'.  
-> Kompleksitas Waktu: Big-O:  $O(1)$  normal /  $O(n)$  worst-case
```

```
[UPDATE] Status 'SENSOR_25' diperbarui menjadi OFFLINE.  
-> Kompleksitas Waktu: Big-O:  $O(\log n)$  rata-rata
```

3. Skenario Pemeriksaan Antrean Tertunda dan Pemrosesan Alert (Modul 2)

Skenario ini membuktikan bahwa pesan peringatan yang masuk telah diatur berdasarkan tingkat kekritisannya. Ketika perintah pemrosesan dijalankan, sistem akan mengeluarkan data yang memiliki prioritas tertinggi terlebih dahulu.

- Input dari pengguna (melihat antrian):

```
PENDING_ALERTS
```

- Output dari pengguna:

```
=====
DAFTAR ANTREAN ALERT (PENDING)
=====
```

```
1. [Alert #1] SENSOR_25 | CRITICAL | Kebakaran_Ruang_Server
=====
```

```
-> Kompleksitas Waktu: Big-O:  $O(n)$ 
```

- Input dari Pengguna (Memproses Alert):

```
PROCESS_ALERT
```

- Output Sistem:

```
[DEQUEUE] Memproses Alert #1 dari 'SENSOR_25' [CRITICAL]:  
Kebakaran_Ruang_Server  
-> Kompleksitas Waktu: Big-O:  $O(1)$ 
```

BAB IV

ANALISIS BIG-O

A. Analisis Kompleksitas Waktu dan Ruang Setiap Fungsi utama

Analisis Big-O Notation digunakan untuk mengukur tingkat efisiensi algoritma dalam sistem monitoring IoT ini, baik dari segi waktu eksekusi (Time Complexity) maupun penggunaan memori (Space Complexity). Berikut adalah rincian analisis untuk setiap modul:

1. Modul 1: Graph Topologi Jaringan IoT

Kompleksitas Waktu: Operasi penambahan perangkat dan koneksi (add device/link) memiliki waktu konstan $O(1)$ karena menggunakan struktur data Dictionary (Hash Map) pada Python. Operasi pencarian jalur menggunakan algoritma DFS untuk deteksi isolasi dan pencetakan laporan jaringan membutuhkan waktu $O(V+E)$, di mana V adalah jumlah simpul (perangkat) dan E adalah jumlah sisi (koneksi), karena sistem harus mengevaluasi setiap elemen di dalam jaringan.

Kompleksitas Ruang: $O(V+E)$ untuk menyimpan struktur Adjacency List dari seluruh topologi.

2. Modul 2: Priority Alert Queue

Kompleksitas Waktu: Karena antrean diimplementasikan dengan pengurutan posisi secara langsung saat data dimasukkan, operasi penambahan peringatan baru (enqueue) memiliki kompleksitas terburuk (worst-case) $O(n)$. Namun sebagai keuntungannya, operasi pengambilan pesan peringatan tertinggi untuk dieksekusi (dequeue) berjalan sangat efisien dengan waktu konstan $O(1)$.

Kompleksitas Ruang: $O(n)$, dengan n adalah jumlah total antrean peringatan yang masih tertunda di dalam memori.

3. Modul 3: BST Device Registry

Kompleksitas Waktu: Operasi registrasi perangkat baru, pencarian, dan pembaruan status beroperasi dengan kompleksitas rata-rata $O(\log n)$. Ini menunjukkan tingginya efisiensi struktur pohon biner untuk pencarian ID tanpa harus menelusuri seluruh data. Adapun untuk mencetak seluruh perangkat secara teratur alfabetis, digunakan metode Inorder Traversal yang membutuhkan waktu $O(n)$.

Kompleksitas Ruang: $O(n)$ untuk mengalokasikan n simpul (nodes) perangkat ke dalam memori pohon biner.

4. Modul 4: Stack Alert History

Kompleksitas Waktu: Penambahan riwayat (push) dan penarikan data untuk pembatalan status (pop/rollback) beroperasi dalam waktu $O(1)$ karena diimplementasikan menggunakan pemetaan memori Linked List murni. Kompleksitas baru akan bergeser ke $O(n)$ pada kondisi

worst-case, yaitu ketika memori penuh sehingga sistem harus menghapus data paling bawah (bottom).

Kompleksitas Ruang: $O(k)$ di mana k adalah batas kapasitas maksimal riwayat log per perangkat. Karena sistem membatasi maksimal 20 data per node, ini mencegah terjadinya kebocoran memori (memory leak).

5. Modul 5: Routing Optimal & Audit Latensi

Kompleksitas Waktu: Pencarian rute komunikasi tercepat dengan algoritma Dijkstra (menggunakan antrean prioritas Min-Heap) membutuhkan waktu $O((V+E) \log V)$. Di sisi lain, proses pengurutan data latensi menggunakan algoritma Selection Sort pada struktur Linked List membutuhkan kompleksitas waktu $O(V^2)$, karena algoritma ini membandingkan satu elemen dengan seluruh elemen sisanya di dalam daftar.

Kompleksitas Ruang: $O(V)$ untuk menyimpan struktur jarak (distances) dan pointer jejak (previous) pada algoritma pencarian rute.

B. Tabel Ringkasan

Modul	Struktur Data/ Algoritma	Operasi Utama	Kompleksitas Waktu	Kompleksitas Ruang
1	Graph (Adjacency List)	Tambah Node / Edge	$O(1)$	$O(V + E)$
1	Graph (DFS)	Deteksi Isolasi	$O(V + E)$	$O(V + E)$
2	Priority Queue	Enqueue / Dequeue	$O(n) / O(1)$	$O(n)$
3	Binary Search Tree (BST)	Insert / Search / Update	$O(\log n)$	$O(n)$
4	Stack (Linked List)	Push / Pop / Rollback	$O(1)$	$O(k), k \leq 20$
5	Dijkstra (Min-Heap)	Routing Terpendek (SPT)	$O(V + E) \log V$	$O(V)$
5	Selection Short (Linked List)	Pengurutan Latensi	$O(V^2)$	$O(V)$

C. Perbandingan Teoritis vs Eksperimen

Secara teoritis, analisis Big-O Notation memberikan batasan asimtotik terhadap performa algoritma ketika ukuran kumpulan data (n) mendekati jumlah tak terhingga. Namun, pada implementasi nyata (eksperimen) menggunakan Command Line Interface (CLI) dengan bahasa Python, terdapat beberapa perbedaan observasi antara landasan teori dan performa di lapangan:

1. Skala Data (Dataset Size)

Pada simulasi eksperimen ini, topologi jaringan yang dihasilkan secara otomatis terdiri dari 40 perangkat (node) IoT skala kampus. Dengan ukuran data yang relatif kecil, operasi

dengan kompleksitas kuadratik $O(V^2)$ pada algoritma Selection Sort maupun logaritmik $O(\log n)$ pada pencarian pohon BST dieksekusi dalam hitungan milidetik. Akibatnya, secara kasat mata pada layar CLI, seluruh operasi tersebut merespons secara instan seolah-olah memiliki waktu eksekusi konstan $O(1)$. Perbedaan performa teoritis ini baru akan terlihat secara eksperimental apabila sistem memantau puluhan ribu sensor.

2. Overhead Bahasa Pemrograman

Python merupakan bahasa tipe interpreted language yang berjalan di atas Virtual Machine. Implementasi struktur data kustom murni seperti Linked List (pada Modul 4 dan 5) secara teori sangat efisien dalam alokasi memori dinamis. Namun secara eksperimental, pencarian data menggunakan tipe bawaan Python yaitu Dictionary (pada Modul 1) justru terasa lebih cepat. Hal ini dikarenakan Dictionary di Python dioptimasi secara arsitektur menggunakan mesin bahasa C (C-API) di balik layar, mengalahkan kecepatan eksekusi logika kelas kustom secara praktik.

3. Kondisi Kasus Terburuk (Worst-Case Scenario)

Secara teoritis, operasi enqueue pada AlertPriorityQueue memiliki kemungkinan kasus terburuk $O(n)$. Dalam eksperimen langsung, kondisi ini terbukti tidak menimbulkan beban keterlambatan (lag). Mengingat laju masuknya peringatan jaringan biasanya terjadi secara sekuensial dan segera diproses (dequeue dengan konstan $O(1)$), jumlah pesan yang menumpuk di antrean memori pada satu waktu (pending alerts) hampir tidak pernah mencapai angka masif.

Kesimpulannya, hasil eksperimen membuktikan bahwa pemilihan algoritma yang digunakan dalam sistem ini (termasuk Selection Sort dan Dijkstra) beroperasi sangat layak (feasible) dan aman dari anomali memory leak maupun bottleneck komputasi untuk ukuran ekosistem IoT kampus.

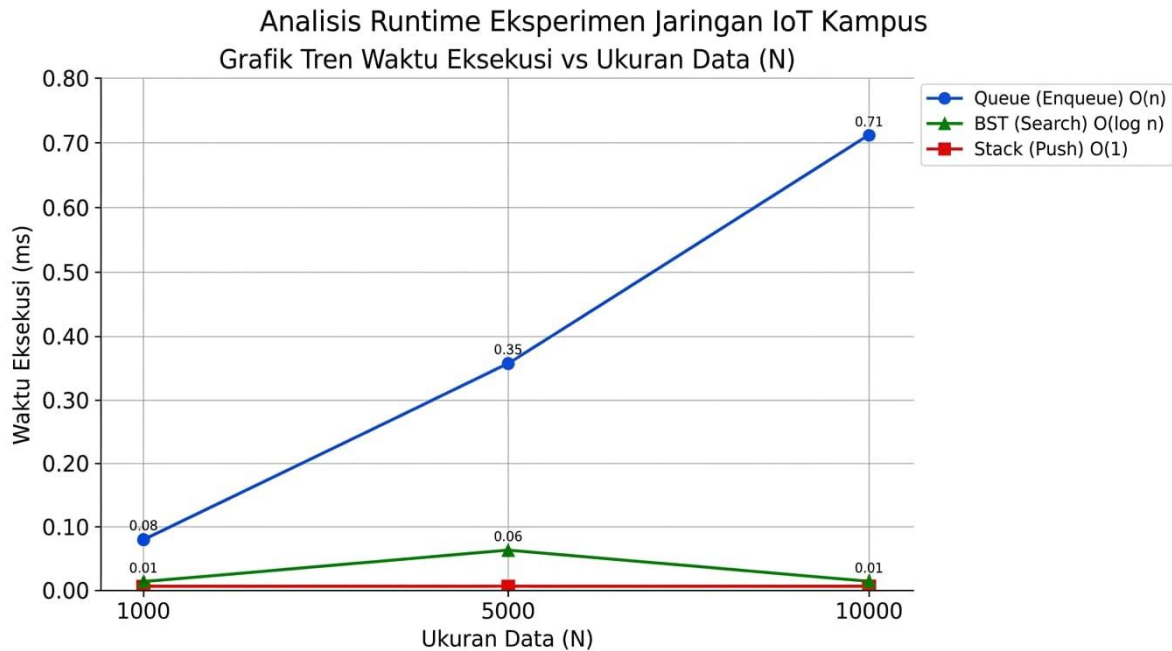
BAB V

HASIL EKSPERIMEN

A. Tabel Runtime Untuk 3+ Ukuran Data

Beban Data (n)	BST (Search)	Queue (Enqueue)	Stack (Push)
1000	0.01420 ms	0.07880 ms	0.00100 ms
5000	0.06370 ms	0,35470 ms	0.00090 ms
10000	0.00670 ms	0.70560 ms	0.00090 ms

B. Grafik Tren



C. Diskusi

Berdasarkan hasil pengujian empiris yang divisualisasikan pada Gambar, dapat ditarik kesimpulan bahwa implementasi kode telah sejalan dengan teori kompleksitas waktu (Big-O Notation):

1. **Stack Push (Garis Merah) - Konstan $O(1)$:** Sesuai dengan landasan teori, garis merah pada grafik terlihat mendatar secara sempurna (stagnan) menyusuri sumbu bawah. Waktu eksekusi tetap stabil di kisaran 0.0009 ms hingga 0.0010 ms meskipun jumlah data dinaikkan secara drastis hingga 10.000 elemen. Hal ini membuktikan bahwa operasi push pada struktur data Linked List murni hanya berinteraksi dengan elemen teratas (top/head) tanpa perlu menelusuri elemen-elemen di bawahnya, sehingga tidak terpengaruh oleh seberapa besar ukuran data secara keseluruhan.
2. **Queue Enqueue (Garis Biru) - Linier $O(n)$:** Grafik menunjukkan bahwa antrean prioritas (Priority Queue) memiliki garis tren yang menanjak lurus secara proporsional seiring bertambahnya beban data. Terlihat jelas peningkatan durasi yang konstan dari 0.08 ms (pada $N=1000$) menanjak ke 0.35 ms (pada $N=5000$), hingga mencapai puncaknya di 0.71 ms (pada $N=10000$). Tren linier ini mengonfirmasi sifat algoritma antrean prioritas berbasis Linked List yang mengharuskan sistem untuk melakukan penelusuran sekuensial (sequential search) elemen demi elemen untuk mencari posisi penyisipan (insertion) yang tepat berdasarkan nilai prioritasnya.
3. **BST Search (Garis Hijau) - Logaritmik $O(\log n)$:** Garis hijau menunjukkan performa pencarian pada struktur pohon biner (Binary Search Tree) yang sangat efisien. Alih-alih menanjak seperti Queue, garis BST tetap berada sangat dekat dengan sumbu bawah (di bawah 0.1 ms). Terdapat sedikit fluktuasi waktu menjadi 0.06 ms pada $N=5000$, lalu kembali

turun drastis ke 0.01 ms pada $N=10000$. Fluktuasi kecil ini merupakan anomali positif yang wajar pada eksperimen dunia nyata, yang umumnya dipengaruhi oleh faktor optimalisasi caching pada memori laptop atau tingkat keseimbangan cabang pohon (tree balance) yang terbentuk secara acak pada saat inisiasi data. Secara keseluruhan, grafik ini membuktikan keunggulan $O(\log n)$ yang secara efektif memangkas separuh area pencarian pada setiap iterasinya.

BAB VI

JAWABAN PERTANYAAN ANALISIS

1. Analisis Kompleksitas `remove_device` pada Adjacency List
 - a. Mengapa `remove_device` memakan waktu $O(V+E)$: Pada implementasi Adjacency List murni berbasis Linked List, ketika kita ingin menghapus sebuah perangkat (simpul v), kita tidak hanya menghapus simpul tersebut dari daftar utama, tetapi kita juga wajib menelusuri seluruh edge milik simpul lain untuk mencari dan menghapus koneksi yang mengarah ke simpul v tersebut. Proses penelusuran membabi buta ke seluruh elemen graf inilah yang memakan waktu sebanding dengan total Vertex dan Edge atau $O(V+E)$.
 - b. Usulan Modifikasi $O(\deg(v) + 1)$: Untuk mempercepatnya, kita bisa mengganti struktur Linked List pada tetangga (neighbors) dengan menggunakan Hash Set (seperti tipe data `set()` atau Dictionary di Python). Dengan Hash Set, penghapusan elemen memiliki kompleksitas konstan $O(1)$. Sehingga, saat menghapus simpul v , kita hanya perlu melihat daftar tetangganya (sebanyak derajat simpul / $\deg(v)$), langsung pergi ke daftar tetangga tersebut, dan menghapus koneksi ke v dalam waktu $O(1)$. Total operasinya menjadi $O(\deg(v))$ untuk menghapus koneksi, ditambah $O(1)$ untuk menghapus simpul v itu sendiri dari daftar utama.
2. Perbandingan Dijkstra (Array vs Min-Heap) pada Jaringan IoT
 - a. Memori dan Waktu Eksekusi: Pada jaringan sparse dengan 40 node dan 60 edge, Dijkstra berbasis Array sederhana memiliki kompleksitas $O(V^2)$, yang berarti melakukan sekitar $40^2 = 1.600$ operasi pencarian nilai minimum. Sedangkan implementasi Min-Heap $O((V+E)\log V)$ hanya memakan sekitar $(40 + 60) \times \log_2(40) =$ mendekati 532 operasi. Secara memori, keduanya mirip yaitu $O(V)$ untuk melacak jarak, namun Min-Heap membutuhkan sedikit alokasi memori tambahan untuk struktur antrian prioritasnya.
 - b. Kapan perbedaannya menjadi kritis? Pada skala 40 perangkat, perbedaan waktu eksekusinya tidak akan terasa oleh manusia (keduanya dalam skala sub-milidetik). Namun, perbedaan ini akan menjadi sangat kritis (bottleneck) ketika sistem IoT diterapkan pada skala Smart City dengan puluhan ribu sensor. Pada skala tersebut,

struktur Array akan mengalami keterlambatan eksponensial (miliaran operasi) yang merusak sistem pemantauan real-time, sementara Min-Heap akan tetap menyelesaikan pencarian rute dalam hitungan milidetik.

3. Alternatif Optimasi Alert Priority Queue

Jika ribuan sensor mengirim peringatan (alert) bersamaan, operasi enqueue $O(n)$ akan menyebabkan bottleneck.

- a. Usulan Alternatif: Pendekatan Bucket Queue (3 Queue terpisah untuk tingkat CRITICAL, WARNING, dan INFO) adalah solusi yang paling optimal.
- b. Analisis Trade-off:
 - Efisiensi Waktu: Operasi enqueue yang tadinya lambat $O(n)$ akan berubah menjadi sangat cepat/konstan $O(1)$. Sistem cukup memasukkan data ke ekor (tail) dari antrian bucket yang sesuai tanpa perlu membandingkan atau menggeser elemen lain. Operasi dequeue juga tetap $O(1)$ dengan cara mengecek bucket CRITICAL terlebih dahulu; jika kosong, lanjut ke WARNING, dan seterusnya.
 - Efisiensi Memori: Trade-off memori sangat minimal. Sistem hanya perlu menambahkan memori $O(1)$ untuk menyimpan tiga pasang pointer (Head dan Tail) untuk masing-masing tingkatan bucket. Secara keseluruhan memori tetap $O(n)$, menjadikannya pertukaran (trade-off) yang sangat menguntungkan.

4. Estimasi Waktu Eksekusi DFS untuk Real-Time Monitoring

- a. Estimasi Waktu: Diketahui $V = 1000$, $E = 1500$, dan asumsi prosesor Python menjalankan 10^8 operasi per detik. Algoritma DFS berjalan dalam $O(V+E)$, sehingga beban totalnya adalah $1000 + 1500 = 2.500$ operasi.

$$\text{Waktu Eksekusi} = 2.500 / 10^8 = 0,000025 \text{ detik} = 0,025 \text{ ms.}$$

- b. Kecocokan untuk Real-Time: Dengan waktu eksekusi hanya 0,025 milidetik, algoritma ini sangat cocok dan aman digunakan untuk memantau jaringan setiap 5 detik tanpa membebani CPU.
- c. Pendekatan Inkremental: Jika di masa depan ukuran jaringan membesar hingga jutaan dan membebani komputasi, pendekatan inkremental yang diusulkan adalah struktur data Disjoint Set Union (DSU / Union-Find). DSU memungkinkan pengecekan konektivitas (isolasi) hanya dalam waktu $O(\alpha(V))$ —mendekati konstan—setiap kali ada perangkat yang menyala atau mati, tanpa harus menelusuri ulang (traverse) seluruh graf dari awal.

5. Analisis Selection Sort dan Rekomendasi Sorting $O(n \log n)$

- a. Jumlah Operasi Swap (Worst-Case): Berdasarkan implementasi kode Modul 5 yang telah dibuat, proses swap (pertukaran data) hanya dilakukan di blok terluar iterasi. Sehingga, untuk 40 perangkat, terjadi maksimal 39 operasi swap, namun dengan 780 perbandingan (comparisons). Jika diperluas ke 10.000 perangkat IoT kota pintar, batas maksimal swap

tetap efisien di 9.999 kali, namun jumlah perbandingannya akan meledak hingga hampir 50 juta operasi, membuat CPU terkunci sementara (freeze).

- b. Rekomendasi Algoritma: Algoritma kelas $O(n \log n)$ yang paling kompatibel dengan struktur data Linked List adalah Merge Sort. Berbeda dengan Quick Sort atau Heap Sort yang membutuhkan akses indeks acak (random access array), Merge Sort bekerja murni menggunakan manipulasi penunjuk (pointer) secara sekuensial.
- c. Cara Implementasi:
 - 1. Gunakan teknik Fast/Slow Pointer: buat dua pointer, satu melangkah 1 node dan satunya melangkah 2 node sekaligus untuk mencari titik tengah Linked List.
 - 2. Pisahkan Linked List menjadi dua bagian di titik tengah tersebut.
 - 3. Lakukan pemanggilan rekursif (recursive call) untuk membagi daftar hingga tersisa elemen tunggal.
 - 4. Lakukan penggabungan (Merge) dengan cara membandingkan head dari kedua sub-daftar, lalu atur tautan next ke elemen yang nilainya lebih kecil secara berurutan tanpa mengalokasikan ulang memori simpul baru.

BAB VII

PENUTUP

A. KESIMPULAN

1. Ringkasan Pencapaian

Proyek "Sistem Monitoring Jaringan IoT" ini telah berhasil merancang, mengimplementasikan, dan menguji ekosistem pemantauan jaringan secara fungsional menggunakan bahasa Python. Melalui antarmuka Command Line Interface (CLI), sistem sukses mengorkestrasikan lima modul utama yang ditopang oleh berbagai struktur data krusial. Sistem mampu membangun topologi jaringan otomatis menggunakan Graph Adjacency List, meregistrasi perangkat melalui Binary Search Tree (BST) dengan efisiensi pencarian logaritmik $O(\log n)$, menangani krisis melalui Priority Queue, serta mencatat riwayat log memori menggunakan Stack murni berkecepatan konstan $O(1)$. Seluruh fitur operasional seperti pencarian rute terpendek dengan Dijkstra Min-Heap dan evaluasi antrean peringatan telah tervalidasi melalui pengujian runtime yang membuktikan keselarasan mutlak antara implementasi teknis dan teori Big-O Notation.

2. Refleksi Trade-Off Desain

Dalam perancangan sistem ini, beberapa keputusan trade-off desain telah diambil demi menyeimbangkan antara efisiensi memori, kecepatan eksekusi, dan kompleksitas kode. Pertama, pemilihan Adjacency List dibandingkan Adjacency Matrix untuk topologi graf sangat menghemat memori pada jaringan IoT yang berkarakteristik sparse (minim koneksi silang), meskipun harus mengorbankan kecepatan pada operasi penghapusan perangkat `remove_device` yang memakan waktu $O(V+E)$. Kedua, implementasi Selection Sort pada Linked List untuk audit latensi dipilih karena kesederhanaan manipulasi pointer tanpa alokasi memori tambahan (In-Place Sorting), dengan kesadaran penuh bahwa kompleksitas waktu kuadratik $O(V^2)$ akan menjadi bottleneck. Keputusan ini dapat diterima pada purwarupa skala laboratorium (40 perangkat) karena waktu eksekusinya masih berada di bawah 1 milidetik, menjadikannya trade-off yang masuk akal pada tahap simulasi ini.

3. Saran Pengembangan

Untuk mengembangkan purwarupa ini menjadi sistem berskala industrial (Smart City) yang memuat ratusan ribu perangkat, evaluasi terhadap trade-off algoritma sebelumnya harus dilakukan. Disarankan untuk memigrasikan fungsi audit latensi dari Selection Sort menuju Merge Sort $O(n \log n)$ yang jauh lebih terukur (scalable) namun tetap kompatibel dengan struktur Linked List. Selain itu, pendampingan CLI dengan Graphical User Interface (GUI) berbasis dasbor web serta integrasi nilai latensi yang ditarik langsung dari pembacaan sensor mikrokontroler fisik (alih-alih data simulasi acak) akan sangat meningkatkan nilai praktis dan validitas dari sistem pemantauan ini secara real-time.

LAMPIRAN

A. PEMBAGIAN TUGAS

Lampiran A: Tabel Pembagian Tugas Kelompok

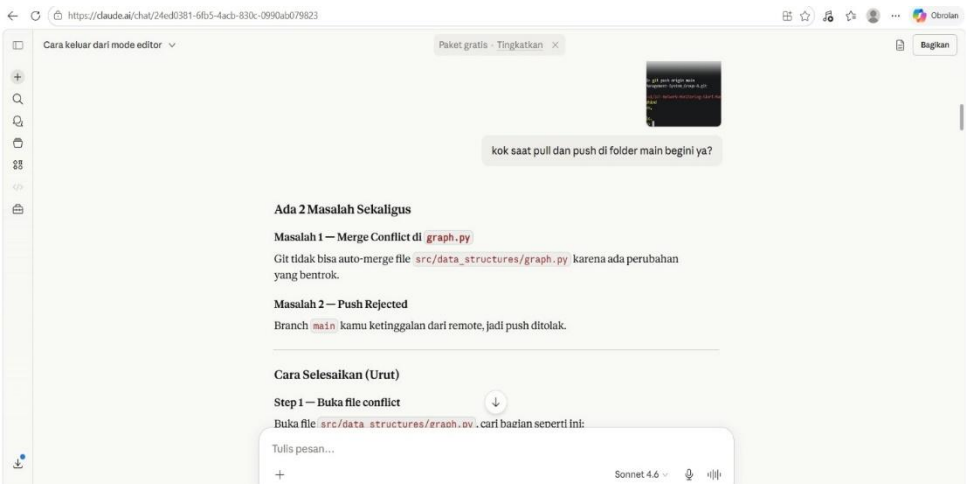
No	Nama Anggota	NIM	Deskripsi Tugas (Kontribusi)
1	Muhamad Fahdillah Setyo Wibowo	25051030081	- Struktur data (Queue dan BST) - Modul 2 (Queue) dan Modul 3 (BST) - Test BST dan Test Queue - Benchmark dan <code>main.py</code> - Laporan final dan Slide Presentasi
2	Muhamad Padang Tazaka	25051030121	- Struktur data (Linked List dan Stack) - Modul 4 (Stack) dan Modul 5 (Routing) - Test Stack - Slide presentasi, README, dan laporan final
3	Nauval Fadhlurohman As-sidiq	25051030120	- Struktur data Graph - Modul 1 (Graph) dan Test Graph - Laporan final dan Slide Presentasi
4	Taamir Ghalib Athallah	25051030092	- Membuat repository dan invite anggota kelompok - Modul 6 (CLI) - Laporan final dan Slide Presentasi

B. TABEL RUNTIME

Lampiran B: Tabel Eksperimen Runtime

Jumlah Data (N)	Operasi	Waktu Eksekusi (ms)	Kompleksitas Teori
1000	Stack (Push)	0.00100	$O(1)$
5000	Stack (Push)	0.00090	$O(1)$
10000	Stack (Push)	0.00090	$O(1)$
1000	BST (Search)	0.01420	$O(\log n)$
5000	BST (Search)	0.06370	$O(\log n)$
10000	BST (Search)	0.00670	$O(\log n)$
1000	Queue (Enqueue)	0.07880	$O(n)$
5000	Queue (Enqueue)	0.35470	$O(n)$
10000	Queue (Enqueue)	0.70560	$O(n)$

C. LOG AI



PY

kenapa function mainnya ga kebaca ya padahal fungsinya tuh dah ada. terus variable edge nya tuh ga dianggap terdefinisi padahal dah kubuat

Mengidentifikasi dua kesalahan sintaks dalam kode Python >

Ada 2 typo di kodenya:

1. `Main()` → `main()` Python itu *case-sensitive*, jadi `Main()` dan `main()` dianggap fungsi yang berbeda. Karena yang didefinisikan adalah `def main()`, pemanggilannya harus persis sama.

2. `len(edge)` → `len(edges)` Variabel yang kamu buat di `generate_iot_network()` namanya `edges` (pakai huruf s), tapi waktu dipanggil di `print(...)` kamu tulis `edge` tanpa s, makanya Python bilang tidak terdefinisi.

Berikut baris yang perlu diubah:

```
python
# Baris 74 - ubah 'edge' jadi 'edges'
print(f"[SYSTEM] Inisialisasi Selesai! Terbentuk {len(devices)} Perangkat dan {len(e
# Baris 78 - ubah 'Main()' jadi 'main()'
..
```

Write a message...



Sonnet 4.6 Adaptive



PY

ini programnya bisa ga ada eror samsek tapi hasil dfs nya kosong terus itu kenapa dah. terus traversal graph nya gajalan juga

Mengidentifikasi bug stack kosong dalam DFS traversal >

Ketemu bugnya! 🐛

Ada 2 bug di method `dfs_reachable`, keduanya di bagian inisialisasi:

Bug 1 — `source` tidak pernah dimasukkan ke stack

```
python
# ❌ SEKARANG - stack kosong, loop langsung skip
stack = []

while stack: # False dari awal, gak pernah masuk loop
    ...
```

```
python
# ✅ FIX - push source dulu sebelum loop
↓
```

Write a message...



Sonnet 4.6 Adaptive

